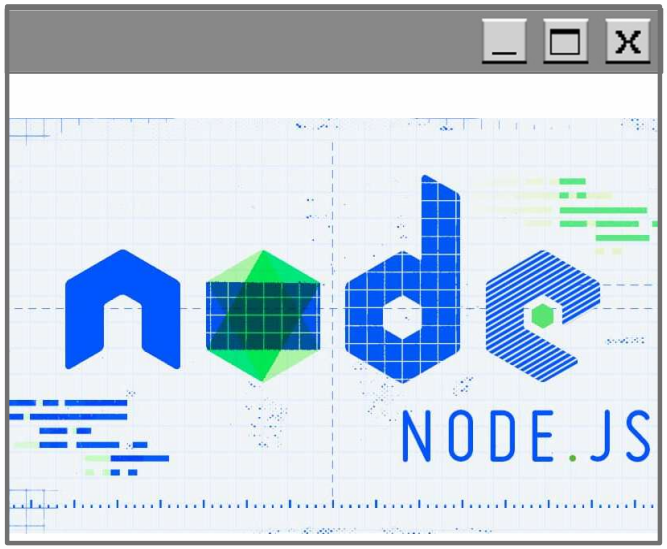
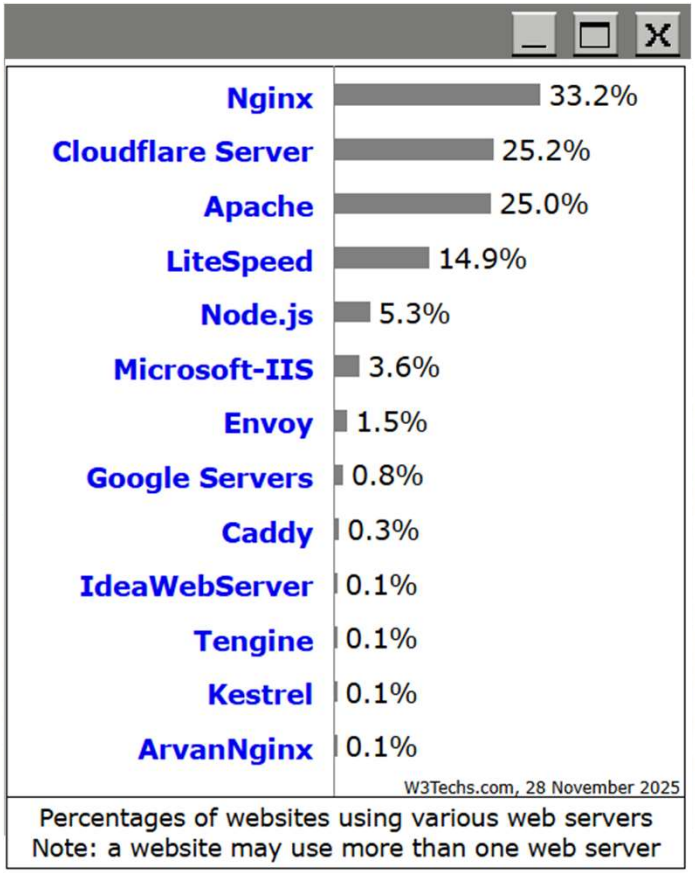
Nikita Niakhai/M25-SNE-01/Unix-based Applications

Node.js as a Unix-based Application:

Evolution, Architecture, Performance, Security



Abstract: **Node.js** – a Unix-focused, event-driven runtime built on V8 and libuv– uses non-blocking I/O and OS-level primitives (epoll/kqueue) to sustain high concurrency with low resource cost. This report analyzes its *architectural foundations*, *performance behavior* as documented in peer-reviewed systems research, and *security posture* shaped by modern vulnerability studies. It also highlights *key limitations*, including *CPU-bound stalls* and *module-system complexity*, and outlines future directions amid *emerging runtimes* such as *Deno and Bun*.



Introduction. Definition

Node.js – is a free, open-source, cross-platform JavaScript runtime environment that lets developers create servers, web apps, command line tools and scripts.

*Currently **Node.js** is one of the most popular (5th place) open source web server on the Internet.*

- One language for frontend and backend (C/S)
- Overcoming Blocking I/O Limitations (Apache ☹)
- High concurrency (C10k → C100k)
- V8 JS engine appeared
- *real use cases – edge, IoT, API gateways, streaming – that need efficient, well-engineered JS runtimes on Unix.*

[01] *https://w3techs.com/technologies/overview/web_server*

[02] *https://nodejs.org/en*

node Evolution. Timeline.

Evolution of Node.js 

2009

Ryan Dahl creates Node.js at **Joyent**, focusing on event-driven I/O for Unix centric runtime for creating web-servers

2010-2014

Early adoption for **high-performance net**works; **Foundation** formed for **governance**

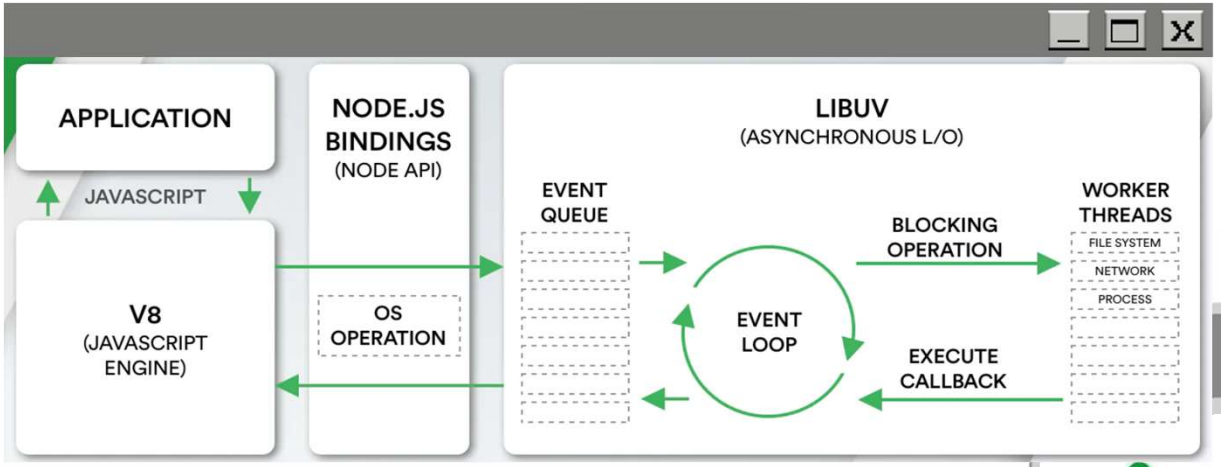
2019

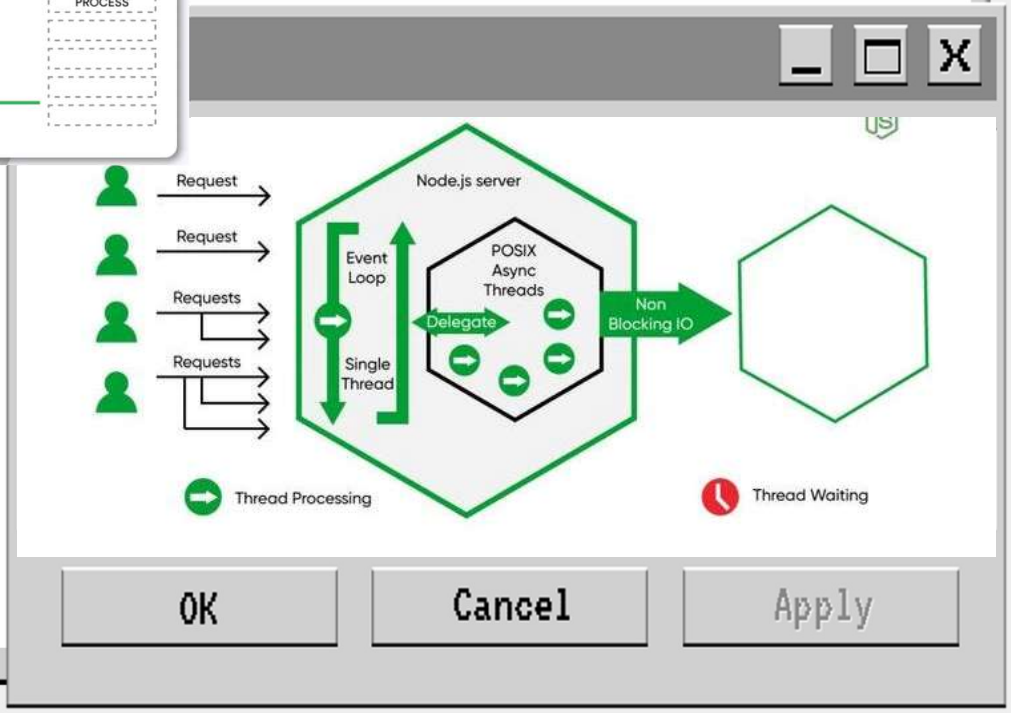
Merges into **OpenJS** Foundation for **open-source** collaboration

2020s

V8/JIT improvements, worker threads, **ESM** maturation, ecosystem **growth**

High-level Process Model





Figures. Diagrams of node.js architecture

OK Cancel Apply

High-level Process Model

Main process

Single-threaded event loop for I/O operations

Event loop integration

Handles callbacks, timers, pending I/O

Clustering

Master process spawns workers for multi-core utilization

Privilege separation

Workers run with limited privileges

Hot reloading

Dynamic updates via cluster restart mechanisms

Event-driven Architecture Deep Dive - Part 1

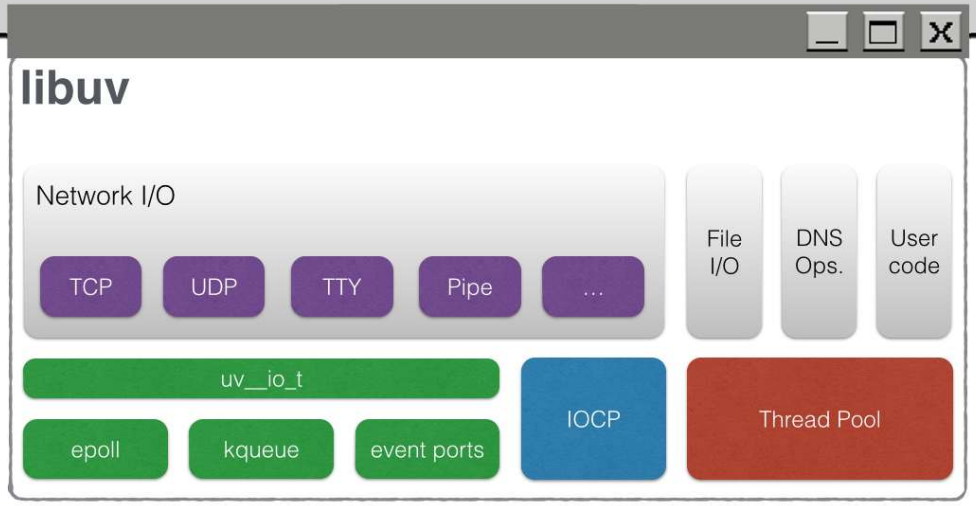


Figure 1. Of libuv architecture

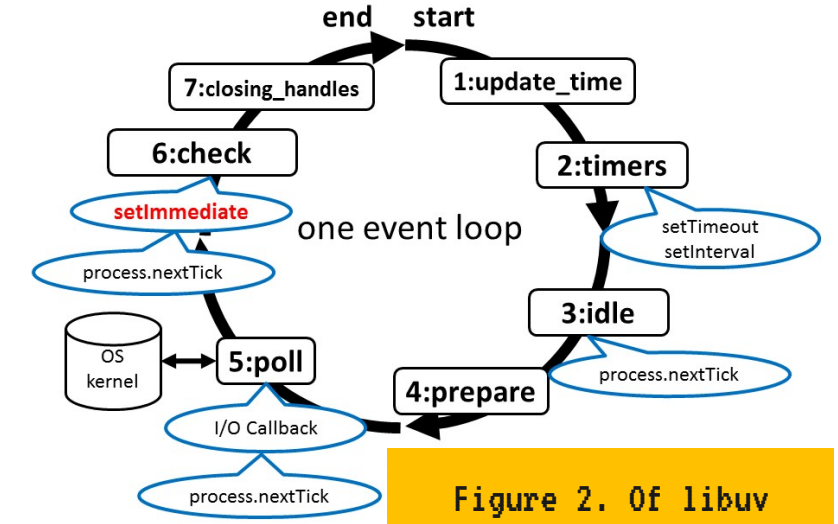


Figure 2. Of libuv event loop

libuv

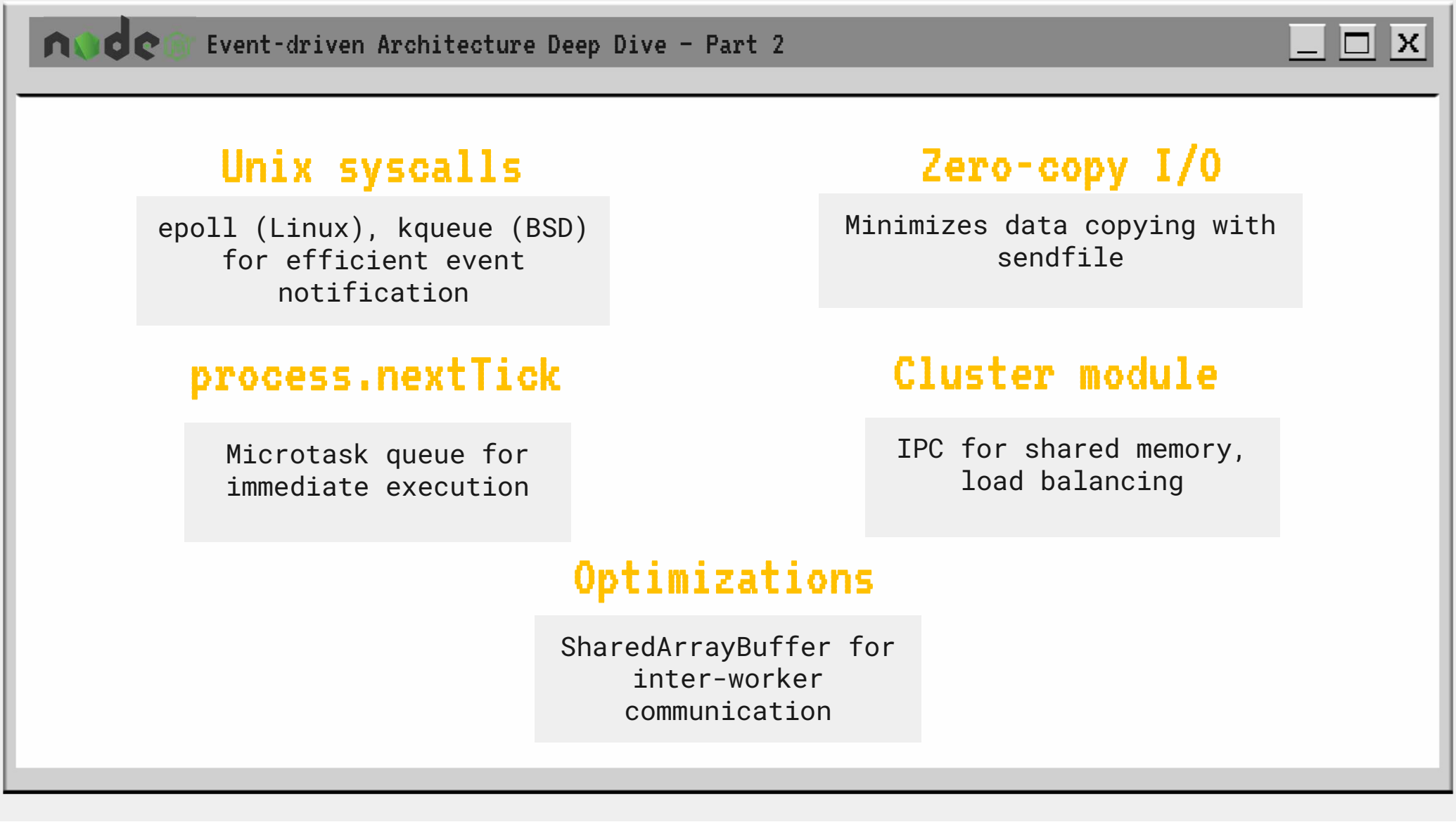
Cross-platform abstraction for asynchronous I/O

Asynchronous I/O

Non-blocking reads/writes using Unix callbacks

Non-blocking everywhere

Avoids thread blocking for scalability


Event-driven Architecture Deep Dive - Part 2
Unix syscalls
epoll (Linux), kqueue (BSD) for efficient event notification
Zero-copy I/O
Minimizes data copying with sendfile
process.nextTick
Microtask queue for immediate execution
Cluster module
IPC for shared memory, load balancing
Optimizations
SharedArrayBuffer for inter-worker communication

Performance Advantages on Unix Systems

C10K+

can handle tens of thousands of clients on modest hardware

Predictable latency, high throughput under overload

Node.js keeps latency steady and predictable—even when overloaded—while pushing high throughput. In contrast, threaded systems can spike in wait times or crash. This is huge for real-world apps like e-commerce sites during sales

Event-driven vs Thread

Studies show SPED servers (like node.js/nginx) outperform thread-/process-based servers on in-memory workloads, scale better, has lower overhead

Clustering extends single-thread benefits

"clustering"—creating a team of worker processes that share the load. It's like one boss (master process) delegating to helpers on Unix, extending the perks of being lightweight while handling more work without the mess of full threading

Performance Advantages on Unix Systems. Part 1

At first, speed (throughput) goes up, but then it peaks and crashes hard, while wait times (latency) explode—like a highway jamming up in rush hour. Event-driven systems (like Node.js) don't do this; they stay steady.

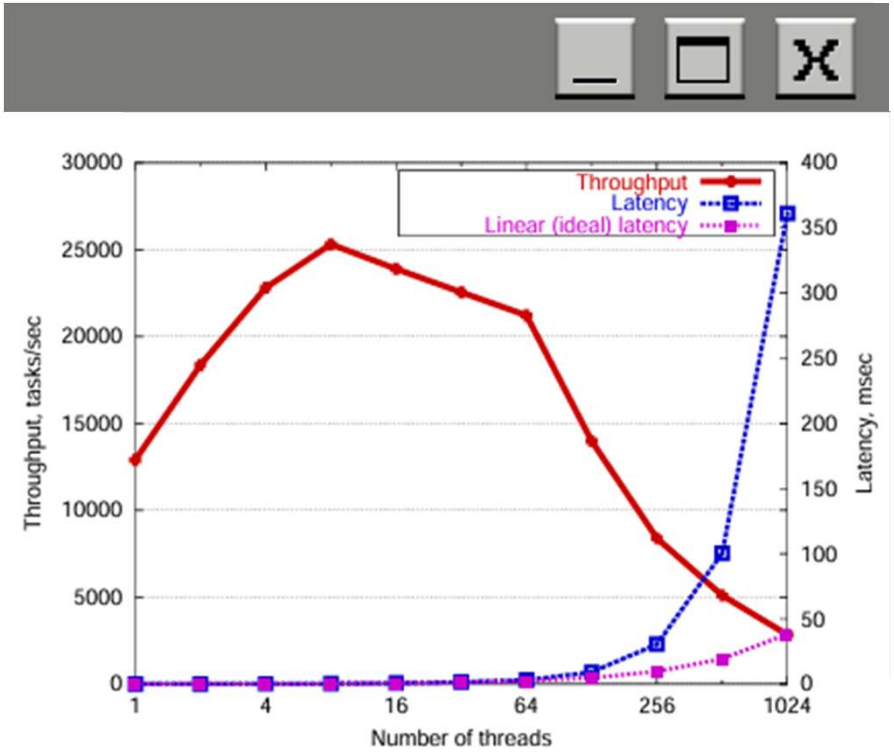
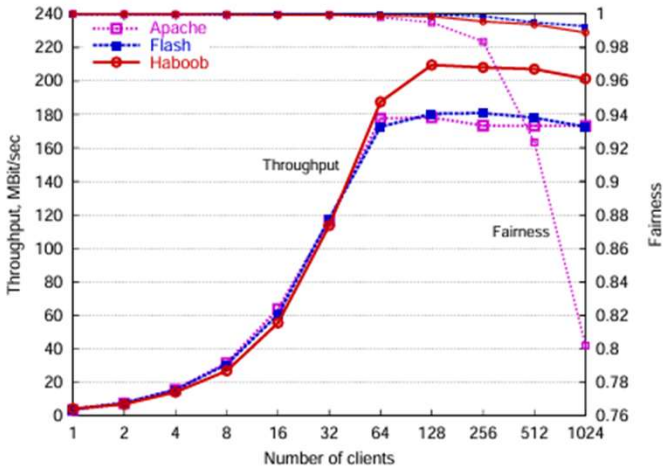
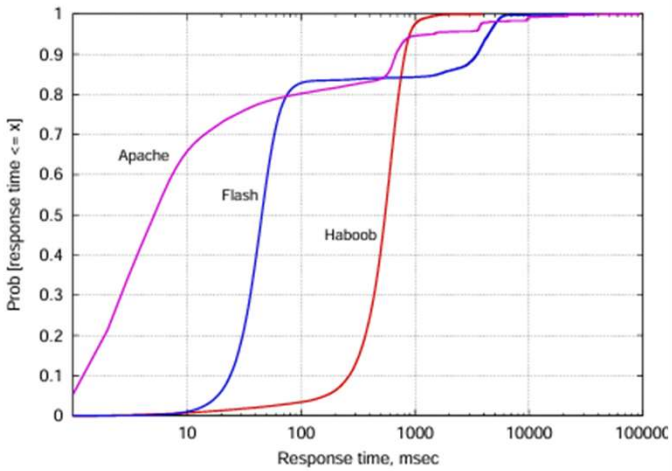


Figure 1. Throughput and latency degradation in threaded servers under increasing load [1]

Comparison of an event-driven server (Haboob, based on SEDA tech that's similar to Node.js) to a threaded one (Apache, an old-school web server). The event-driven one keeps pumping out data at over 200 Mbps (megabits per second) even with 1,000+ users, and it's "fair" (treats all requests equally). Apache drops off sharply. **It's proof that event-driven wins for busy Unix servers.**



(a) Throughput vs. number of clients



(b) Cumulative distribution of response time for 1024 clients

	256 clients				*1024 clients*			
Server	*Throughput*	*RT mean*	*RT max*	*Fairness*	*Throughput*	*RT mean*	*RT max*	*Fairness*
Apache	173.36 Mbps	143.91 ms	27953 ms	0.98	173.09 Mbps	475.47 ms	93691 ms	0.80
Flash	180.83 Mbps	141.39 ms	10803 ms	0.99	172.65 Mbps	665.32 ms	37388 ms	0.99
Haboob	208.09 Mbps	112.44 ms	1220 ms	0.99	201.42 Mbps	547.23 ms	3886 ms	0.98

Figure 2. Throughput and latency degradation in threaded servers under increasing load [1]

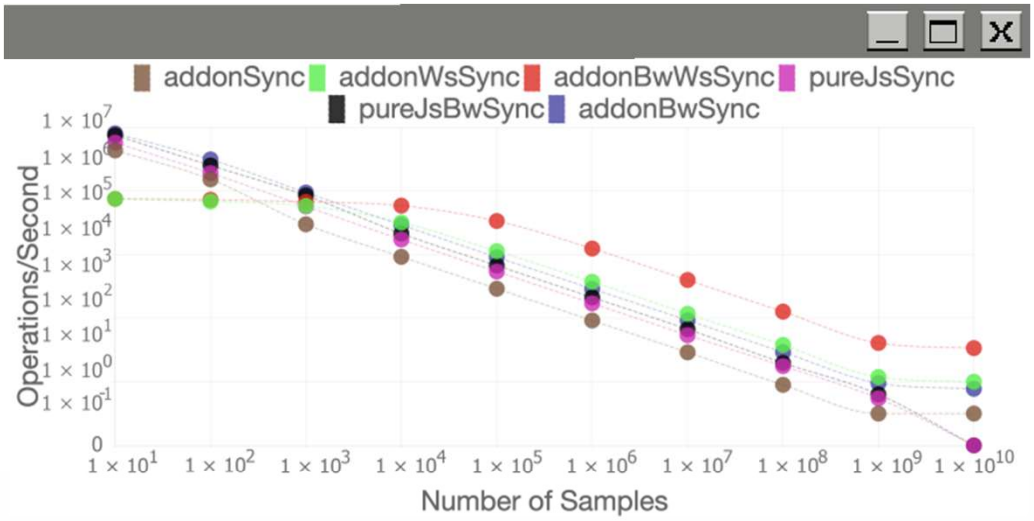


Figure 1. Average operations per second of synchronous implementations in Node.js

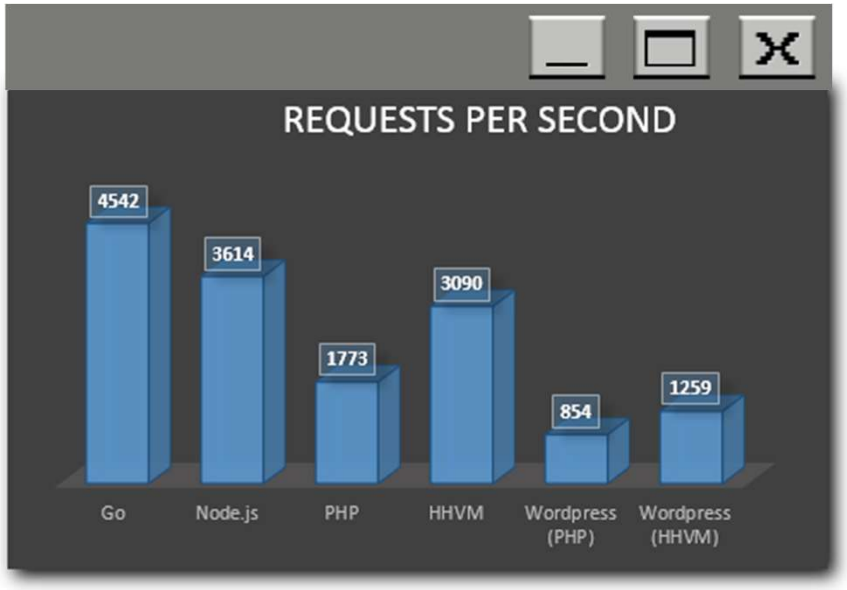


Figure 2. Benchmark comparison for request per second handling capability for different back-end solutions

Security 

- **Process isolation**: Clustering forks multiple worker processes; crashes or breaches in one don't affect others, boosting reliability on Unix.
- **Sandboxing**: VM module runs untrusted code in isolated contexts, blocking access to global scope and limiting malicious damage.
- **Privilege limitation**: Drops root privileges after binding ports, running as low-privilege user to reduce exploit risks.
- **Mitigation**: Object.freeze seals prototypes, stopping attackers from injecting properties that cause DoS or code execution.
- **DoS resistance**: Asynchronous event loop keeps server responsive, preventing slow clients from blocking or exhausting resources.
- **Error handling**: Domains and try-catch group async operations, catching errors centrally to avoid process crashes.

Current limitations

Weaknesses

Single-threaded model creates CPU bottlenecks for intensive tasks; callback complexity ("callback hell") complicates code readability and maintenance in large applications.

Integrations/Research

Rust and WebAssembly integrations enable hybrid apps, significantly boosting CPU-bound performance beyond pure JavaScript limitations (Kyriakou, 2022).

Competition

Deno provides secure-by-default permissions and native TypeScript; Bun delivers superior startup speed and built-in tooling, challenging Node.js in performance and developer experience.

Table of references

[1] **Welsh**, M., Culler, D., & Brewer, E. "**SEDA: An Architecture for Well-Conditioned, Scalable Internet Services.**", 2001: https://www.sosp.org/2001/papers/welsh.pdf

[2] **von Behren**, R., Condit, J., & Brewer, E. "**Why Events Are a Bad Idea (for High-Concurrency Servers.**" Proceedings of the 9th Workshop on Hot Topics in Operating Systems (HotOS), 2003: http://bantha.org/~jcondit/threads-hotos-2003.pdf

[3] **Pai**, V. S., Druschel, P., & Zwaenepoel, W. "**Flash: An Efficient and Portable Web Server.**" **Proceedings of the USENIX Annual Technical Conference** (ATC), 1999: https://www.usenix.org/legacy/event/usenix99/full_papers/pai/pai.pdf

[4] **Tilkov**, S., & Vinoski, S. "**Node.js: Using JavaScript to Build High-Performance Network Programs.**" IEEE Internet Computing, 14(6), 80-83, 2010: https://www.scribd.com/document/948489121/0-Node-js-Using-JavaScript-to-Build-High-Performance-Network-Programs

[5] **Lei**, Y., Liu, J., Yu, Q., & Cao, B. "**Performance Comparison and Evaluation of Web Development Technologies in PHP, Python, and Node.js.**" Proceedings of the IEEE 17th International Conference on Computational Science and Engineering (CSE), 2014: https://www.researchgate.net/publication/286594024_Performance_Comparison_and_Evaluation_of_Web_Development_Technologies_in_PHP_Python_and_Nodejs

[6] **Kyriakou**, K. I. D., & Tselikas, N. D. "**Complementing JavaScript in High-Performance Node.js and Web Applications with Rust and WebAssembly.**" Electronics, 11(19), 3217, 2022: https://www.mdpi.com/2079-9292/11/19/3217

[7] **Koishybayev**, I., & Kapravelos, A. "**Mininode: Reducing the Attack Surface of Node.js Applications.**" Proceedings of the 23rd International Symposium on Research in Attacks, Intrusions and Defenses (RAID), 2020: https://www.usenix.org/system/files/raid20-koishybayev.pdf

[8] **Li**, S., Kang, M., Hou, J., & Cao, Y. "**Mining Node.js Vulnerabilities via Object Dependence Graph and Query.**" Proceedings of the 31st USENIX Security Symposium (USENIX Security), 2022: https://www.usenix.org/system/files/sec22summer_li-song.pdf

Thank you

Do you have any questions?

Node.js as Unix-based Application:

Evolution, Architecture, Performance, Security